Integración y estabilización de PharmaSoft

Informe de la práctica de la sesión 14

Estudiante. Brandon Erly Pacho Mamani

Curso. Lenguaje de Programación II

Docente. Benjamin David Reyna Barreto

Ejecución. 12 y 13 de noviembre de 2026, hora de Lima

Rama de trabajo. feature/integracion-estabilizacion-pacho

La práctica comprobó la paginación del catálogo, la integración del flujo de ventas y el registro de operaciones en la bitácora. Se utilizó un laboratorio local con Oracle, Redis, Spring Boot y Angular.

Comprobación	Resultado de la práctica
Volumen de ensayo	5 000 productos y 20 000 ventas del lote sintético
Matriz de integración	8 de 8 casos cumplidos
Playwright	7/7 · 7/7 · 7/7. Total 21/21 en 23,2 s, sin reintentos
Incidencias	4 registradas y 4 cerradas, incluida INC-04 en esta ejecución

P00 Verificación del entorno

Servicios del laboratorio

Objetivo. Comprobar que la aplicación, la API, la base de datos y la caché están listas para las pruebas.

Procedimiento. Con Oracle local en ejecución, inicié el laboratorio desde PowerShell. El lanzador conectó el backend al esquema PHARMA_S14 y comprobó Redis y la aplicación servida por Nginx.

Resultado observado. El backend quedó UP, Redis devolvió PONG y la aplicación respondió HTTP 200 en el puerto 18085.



Figura 1. Salida de PowerShell con el estado de los servicios.

Comprobación directa del backend

Objetivo. Verificar la API por su puerto directo antes de probar las rutas de negocio.

Procedimiento. Envié GET /actuator/health en Postman al puerto 18084 y revisé el estado HTTP y el campo status del JSON.

Resultado observado. La API devolvió HTTP 200 con status UP. La respuesta también muestra los grupos liveness y readiness.

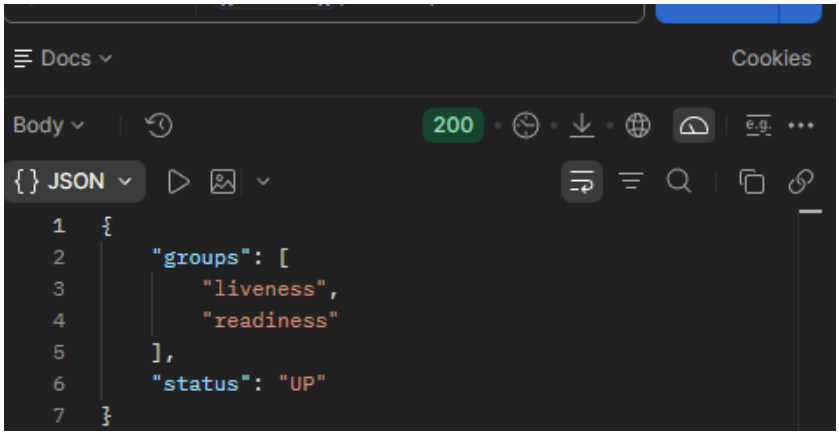


Figura 2. Respuesta HTTP 200 de Actuator en Postman.

P00 Inicio de sesión por rol

Administrador y vendedor

Objetivo. Preparar sesiones separadas para comparar los permisos de administración y de venta.

Procedimiento. En el entorno local de Postman envié POST /api/v1/auth/login con las cuentas de ensayo admin_s14 y vendedor_s14. Revisé los roles del cuerpo y las aserciones de cada solicitud.

Resultado observado. Ambos inicios devolvieron HTTP 200 y dos aserciones aprobadas. El cuerpo identifica los roles ADMIN y VENDEDOR.

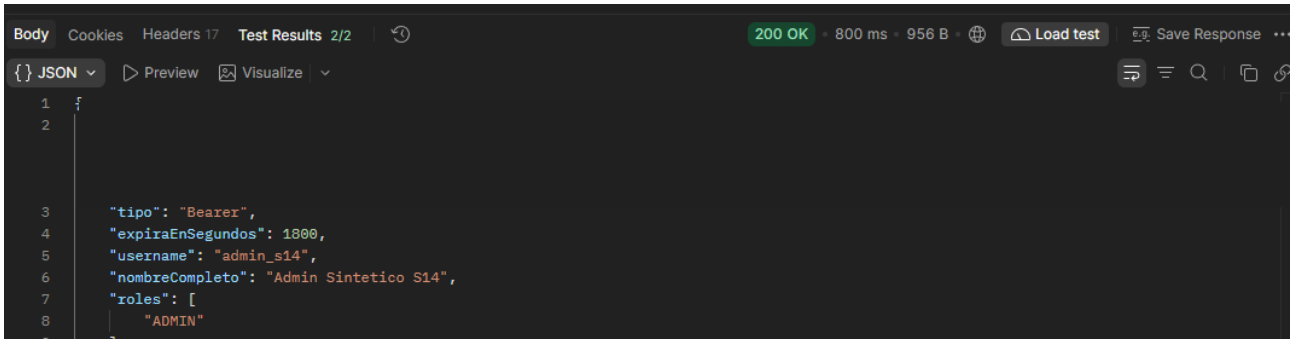


Figura 3. Inicio de sesión del administrador y rol ADMIN. Se ocultó el token.

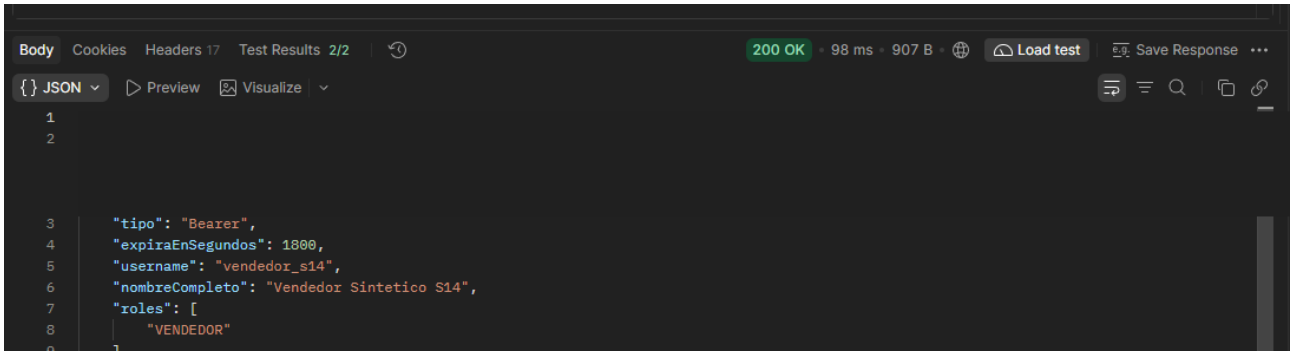


Figura 4. Inicio de sesión del vendedor y rol VENDEDOR. Se ocultó el token.

P01 Línea base de paginación

Medí las solicitudes con Postman y registré el estado, el tiempo y el tamaño en las unidades que muestra la aplicación. Las mediciones corresponden a la ejecución del 12 de noviembre de 2026.

ID	Consulta	HTTP	Tiempo ms	Tamaño
P01-01	Productos de 20 en 20	200	362	1,07 KB
P01-02	Productos de 100 en 100	200	68	2,29 KB
P01-03	Última página de productos	200	61	1,2 KB
P01-04	Tamaño de página 500	409	34	682 B
P01-05	Ventas por rango de fechas	200	117	931 B
P01-06	Orden por categoriaNombre	409	30	658 B

P01-07 Conteo en Oracle

Objetivo. Verificar el volumen de datos que sustenta las mediciones de paginación.

Procedimiento. Desde PowerShell ejecuté SQL*Plus en el contenedor oracle-local. Cambié la sesión a FREEPDB1 y consulté COUNT(*) en las tablas productos y ventas de PHARMA_S14, con el filtro creado_por = s14-volumen.

Resultado observado. El lote sintético contiene 5 000 productos y 20 000 ventas. El filtro permite separar ese volumen de los registros generados durante la matriz manual.

```
PS C:\Windows\system32> @'
>> alter session set container=FREEPDB1;
>> select count(*) productos_carga from PHARMA_S14.productos where creado_por = 's14-volumen';
>> select count(*) ventas_carga from PHARMA_S14.ventas where creado_por = 's14-volumen';
>> exit
>> '@ | docker exec -i oracle-local sqlplus -s / as sysdba

Session altered.


PRODUCTOS_CARGA
---------------
           5000


VENTAS_CARGA
------------
       20000

PS C:\Windows\system32>
```

Figura 5. Consultas SQL y conteos de productos y ventas del lote sintético.

SQL de paginación. El registro de Hibernate muestra OFFSET y FETCH para limitar la consulta de productos, junto con COUNT para calcular el total. El orden estable combina nombre e ID.

P01 Primera página de Productos

P01-01 Tamaño de 20

Objetivo. Medir una consulta con el tamaño de página que usa habitualmente el catálogo.

Procedimiento. Con la sesión ADMIN, envié GET /api/v1/productos con pagina=0 y tamanio=20. Postman comprobó los metadatos, el contenido de la página y que existieran al menos 5 000 registros.

Resultado observado. HTTP 200 en 362 ms y 1,07 KB. Las tres aserciones aprobaron, incluida la que comprueba las 20 filas.

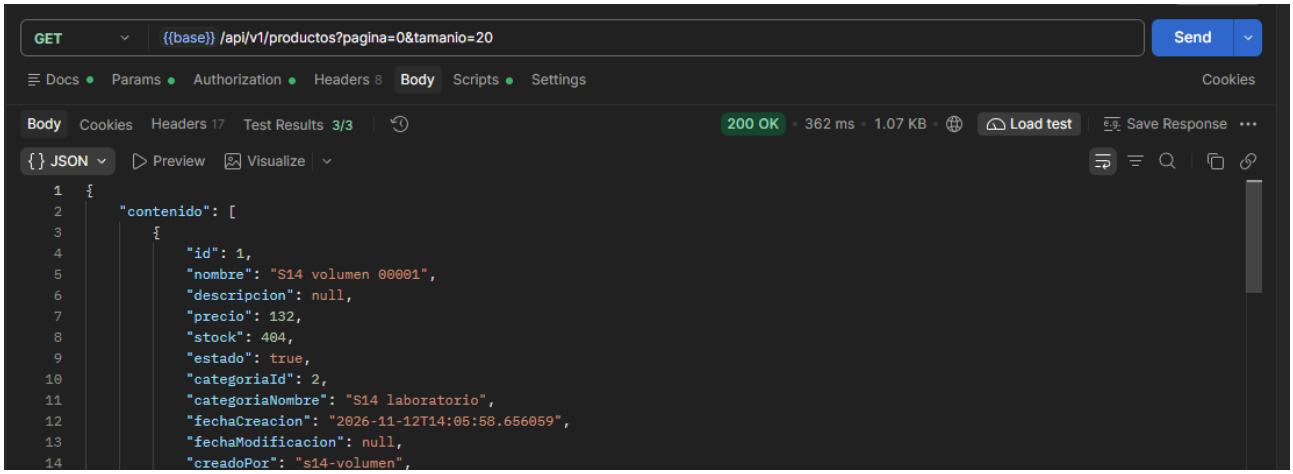


Figura 6. Primera página con tamaño 20. Se muestran la solicitud, las métricas y el inicio del JSON.

P01-02 Tamaño de 100

Objetivo. Comprobar el tamaño máximo admitido y registrar el tamaño de la respuesta.

Procedimiento. Repetí la consulta de la página 0 con tamanio=100. Comprobé que el arreglo contenido devolviera exactamente 100 registros.

Resultado observado. HTTP 200 en 68 ms y 2,29 KB, con tres aserciones aprobadas. La respuesta fue mayor que la de 20 filas. Los tiempos corresponden a una solicitud por tamaño.

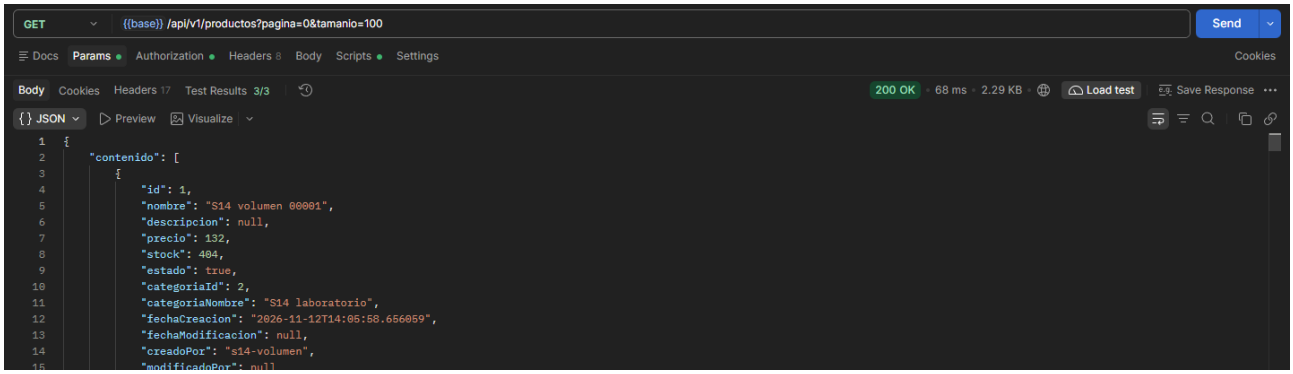


Figura 7. Primera página con tamaño 100 y respuesta HTTP 200.

P01 Última página y límite de tamaño

P01-03 Última página

Objetivo. Comprobar el último bloque de registros con el total del catálogo en ese momento.

Procedimiento. Postman guardó totalPaginas menos uno a partir de P01-01. Envié ese índice como pagina con tamanio=20. La colección comprobó que el bloque tuviera registros y que el indicador ultima fuera verdadero.

Resultado observado. HTTP 200 en 61 ms y 1,2 KB. Las tres aserciones confirmaron la última página.

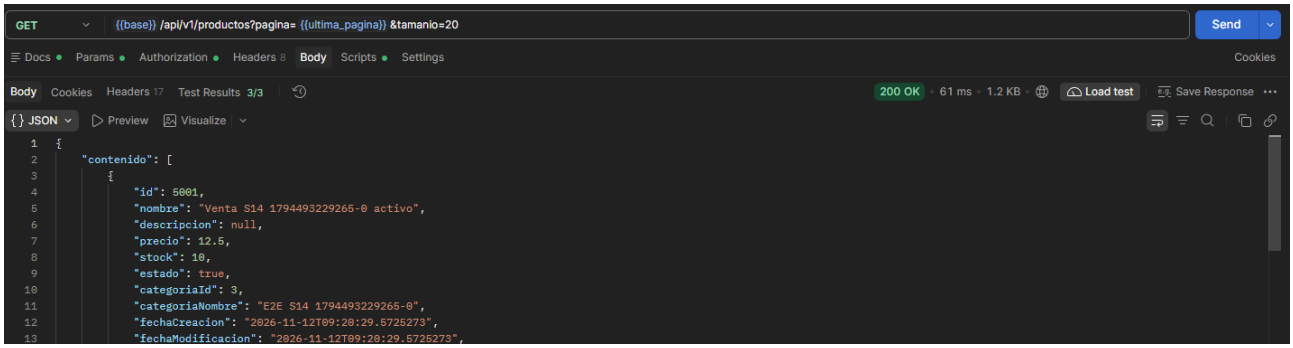


Figura 8. Respuesta y aserciones de la consulta de la última página.

P01-04 Límite de tamaño

Objetivo. Verificar que la API rechaza un tamaño fuera del rango admitido.

Procedimiento. Envié GET /api/v1/productos con pagina=0 y tamanio=500. Revisé el código, el mensaje de validación y el identificador de seguimiento.

Resultado observado. HTTP 409 en 34 ms y 682 B. El mensaje delimita el tamaño entre 1 y 100. Las dos aserciones aprobaron.



Figura 9. Rechazo HTTP 409 para un tamaño de 500 filas.

P01 Ventas por fechas y orden no permitido

P01-05 Ventas por rango

Objetivo. Comprobar que la búsqueda combina filtros de fechas con paginación del servidor.

Procedimiento. Consulté GET /api/v1/ventas/buscar con el rango del 1 al 12 de noviembre de 2026, pagina=0 y tamanio=10. La colección comprobó los metadatos y que las fechas de las ventas devueltas pertenecieran al intervalo.

Resultado observado. HTTP 200 en 117 ms y 931 B, con tres aserciones aprobadas.

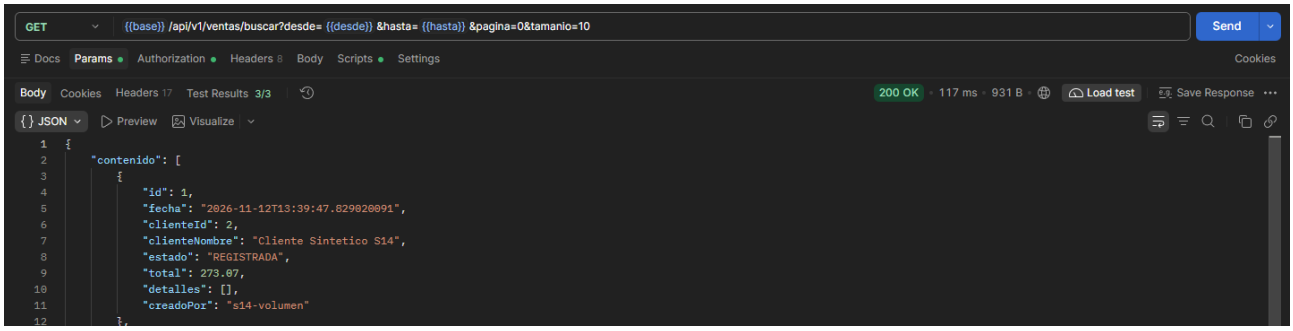


Figura 10. Respuesta paginada del filtro por fechas.

P01-06 Campo de orden rechazado

Objetivo. Verificar que el backend valida el campo de ordenamiento solicitado por el cliente.

Procedimiento. Envié GET /api/v1/productos con pagina=0, tamanio=20 y ordenarPor=categoriaNombre. Revisé el motivo devuelto y el identificador de seguimiento.

Resultado observado. HTTP 409 en 30 ms y 658 B. La API identificó categoriaNombre como campo no permitido y las dos aserciones aprobaron.

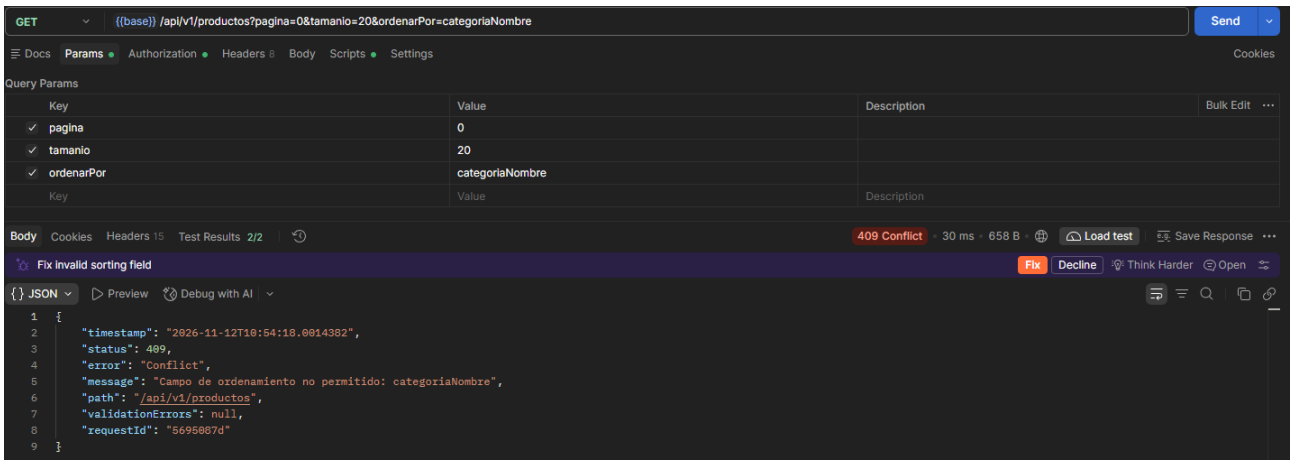


Figura 11. Parámetro fuera de la lista permitida y mensaje de error.

P02 Paginación de Productos

P02-01 Página siguiente

Objetivo. Comprobar que al navegar se solicita otra página al servidor y se actualiza la URL.

Procedimiento. Abrí Productos con DevTools en Red y filtro Fetch/XHR. Desde la primera página pulsé Página siguiente y revisé la solicitud GET del catálogo.

Resultado observado. La interfaz mostró Página 2 de 251. La URL y la solicitud usaron pagina=1 porque la API cuenta desde cero. El GET devolvió HTTP 200.

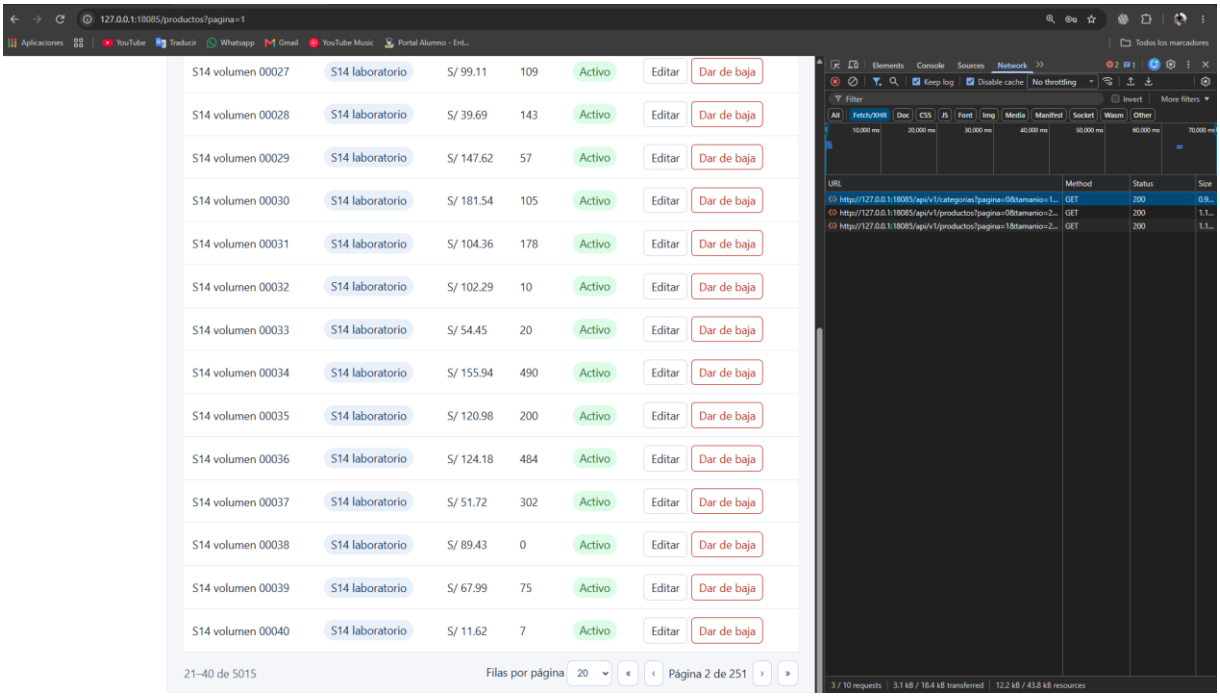


Figura 12. Segunda página de Productos con la consulta en la pestaña Red.

P02-02 Recarga

Objetivo. Comprobar que la posición del catálogo se recupera desde la URL.

Procedimiento. Con la segunda página visible pulsé F5 y revisé la dirección, el paginador y el registro de Red.

Resultado observado. Se conservó pagina=1 y se volvió a mostrar Página 2 de 251.

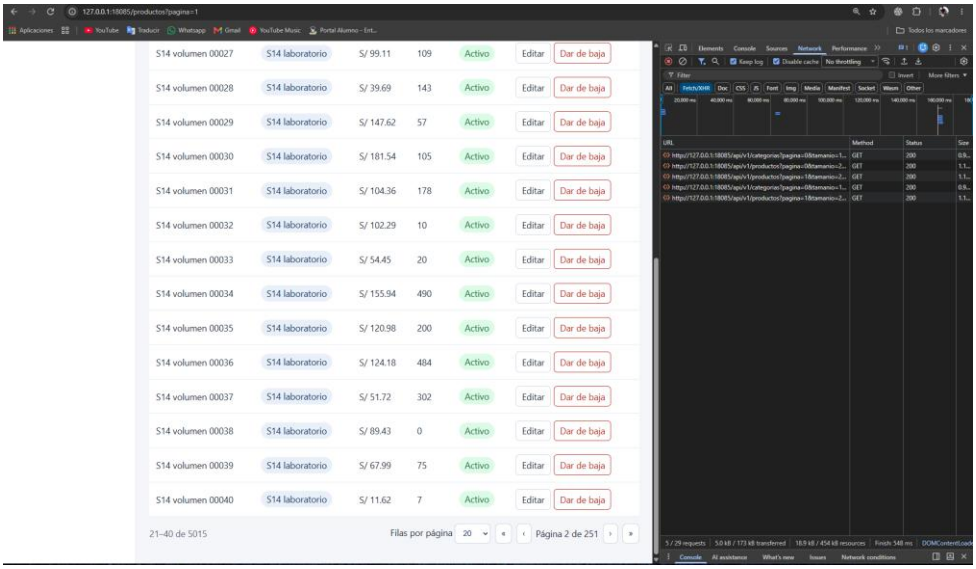


Figura 13. Paginación conservada después de recargar.

P02 Cambio de tamaño

P02-03 Reinicio de la página

Objetivo. Verificar que cambiar la cantidad de filas reinicia la navegación con el tamaño elegido.

Procedimiento. Desde la segunda página cambié Filas por página de 20 a 50. Revisé la dirección, el paginador y los encabezados de la solicitud GET en DevTools.

Resultado observado. La vista volvió a Página 1 de 101. La URL quedó con pagina=0 y tamanio=50. El GET devolvió HTTP 200 con el orden por nombre ascendente.

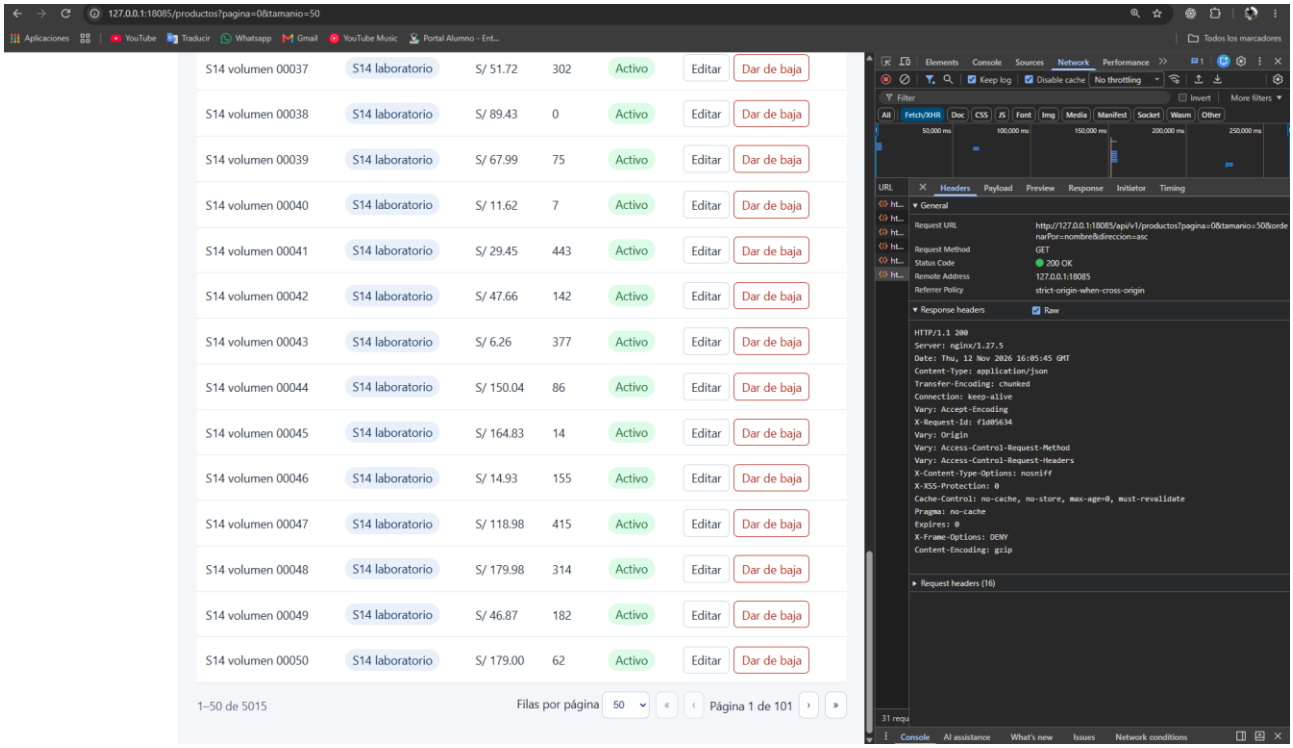


Figura 14. Tamaño 50, página 1 y detalle de la solicitud en DevTools.

P02 Ordenamiento por precio

P02-04 Precio ascendente

Objetivo. Comprobar que el orden por precio se aplica en el servidor y reinicia la página sin cambiar el tamaño elegido.

Procedimiento. Con 50 filas por página, pulsé el encabezado Precio. Revisé la flecha de la columna, los primeros importes y la solicitud GET en Red.

Resultado observado. La URL muestra pagina=0, tamanio=50, orden=precio y dir=asc. El GET devolvió HTTP 200 con ordenarPor=precio y direccion=asc. La tabla muestra Precio ↑ y empieza con S/ 1,01, S/ 1,12 y S/ 1,15.

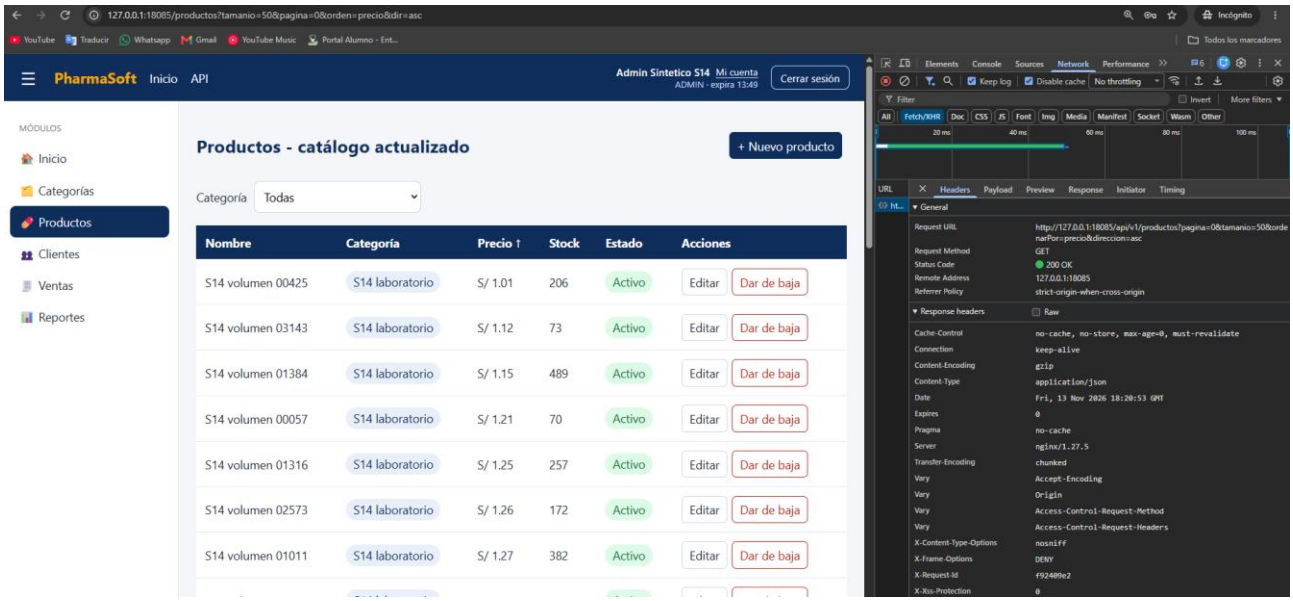


Figura 15. Orden ascendente por precio, parámetros de la URL y respuesta HTTP 200 en DevTools.

Matriz de integración I-01 Cliente

Cliente nuevo en el buscador de Ventas

Objetivo. Verificar que un cliente recién registrado está disponible en Nueva venta.

Procedimiento. Como vendedor_s14, registré el cliente de ensayo desde Clientes. Después abrí Nueva venta, busqué el apellido implementacion y seleccioné el resultado.

Resultado observado. El cliente de ensayo quedó asociado al formulario. La evidencia muestra el nombre seleccionado en el bloque Cliente.



Figura 16. Cliente seleccionado en Nueva venta. Se ocultó el DNI.

Matriz de integración I-02 Catálogo

M00 Preparación del catálogo

Objetivo. Preparar tres registros que permitan distinguir entre estado y disponibilidad.

Procedimiento. Como administrador creé los productos en la categoría S14 Aula 0915 con el mismo precio, S/ 12,50. En Productos filtré por esa categoría.

Resultado observado. El catálogo mostró un producto activo con 10 unidades, otro activo con stock 0 y un tercero inactivo con 10 unidades.



Figura 17. Los tres productos del ensayo y sus estados.

I-02 Productos ofrecidos en la venta

Objetivo. Comprobar que para ofrecer un producto se requieren estado activo y existencias.

Procedimiento. Desde la sesión del vendedor busqué el catálogo de ensayo en el selector de Nueva venta y comparé las opciones con los tres registros de M00.

Resultado observado. El selector ofreció únicamente S14 Aula 0915 activo, con 10 unidades a S/ 12,50. La preparación permitió comprobar las dos condiciones por separado.

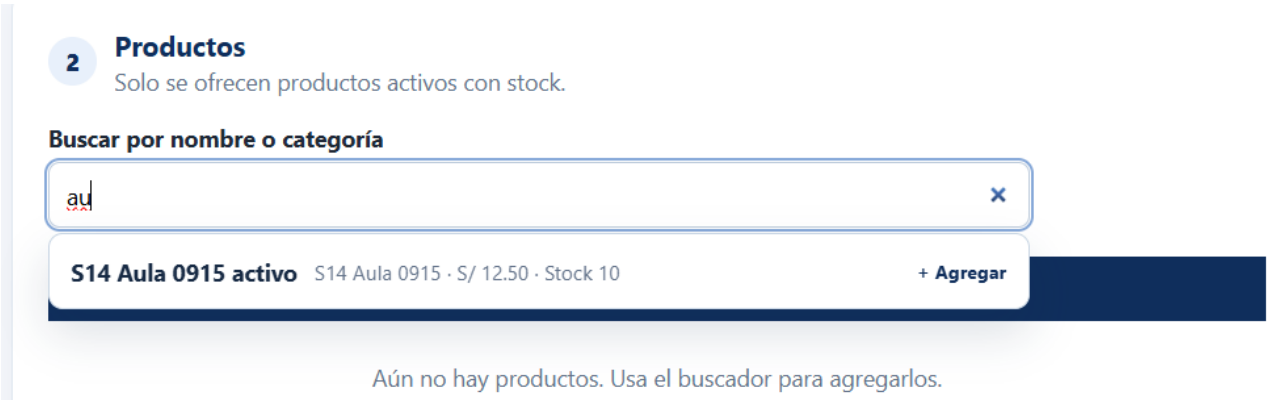


Figura 18. El buscador ofrece únicamente el producto activo con stock.

Matriz de integración I-03 Confirmación

Venta de dos unidades

Objetivo. Verificar que la confirmación registra una venta con las unidades elegidas y el importe calculado.

Procedimiento. Consulté el producto 5016 en Postman antes de vender. En Nueva venta seleccioné al cliente del ensayo, agregué dos unidades a S/ 12,50, revisé el total y confirmé. Inspeccioné el POST /api/v1/ventas en DevTools.

Resultado observado. El stock inicial era 10. El POST respondió HTTP 201, con venta 22221 y total S/ 25,00. El registro del backend asocia esa respuesta con X-Request-Id ce0c94d5.

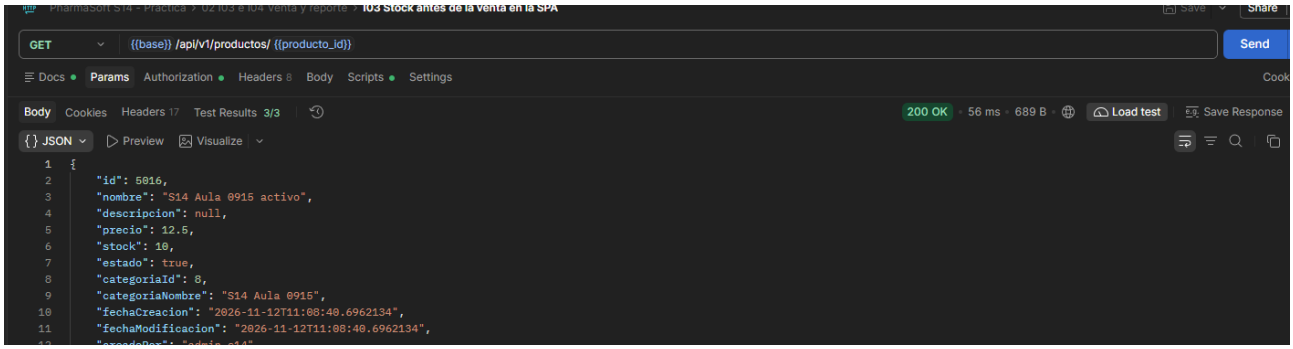


Figura 19. Stock inicial del producto 5016 en Postman.

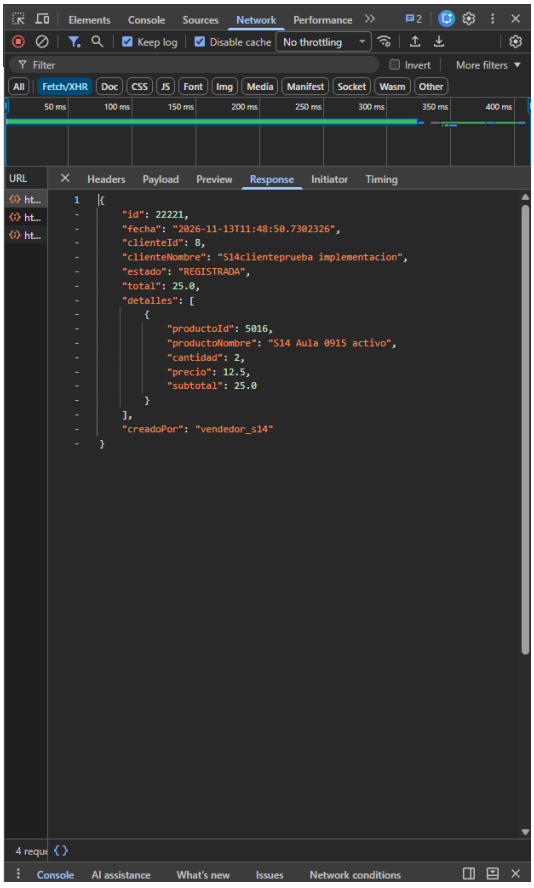


Figura 20. Respuesta de la API con el ID 22221 y el detalle de la venta.

Matriz de integración I-03 Comprobante

Comprobante generado

Objetivo. Comprobar que la aplicación abre el comprobante que corresponde a la venta creada.

Procedimiento. Tras confirmar, revisé el número de comprobante, el autor y el detalle en la pantalla. Consulté también GET /api/v1/ventas/22221 en Postman.

Resultado observado. La pantalla y la API coinciden en el registro de vendedor_s14, las 2 unidades y el total de S/ 25,00. DevTools conserva el POST con HTTP 201.

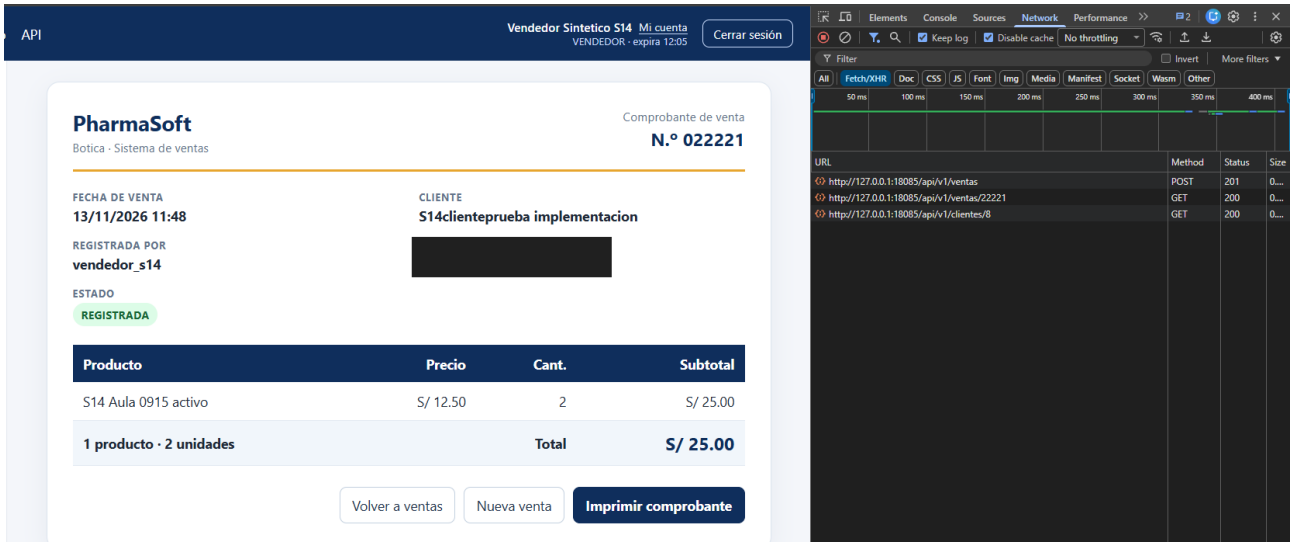


Figura 21. Comprobante de la venta 22221 y respuesta HTTP 201. Se ocultó el DNI.

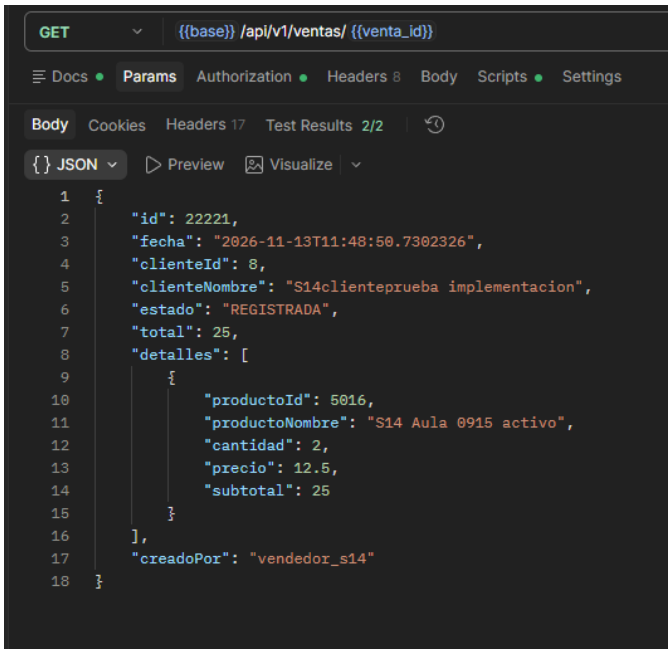


Figura 22. Consulta del comprobante en Postman con dos unidades y total de S/ 25,00.

Matriz de integración I-03 Stock

Descuento de existencias

Objetivo. Verificar que el catálogo refleja el stock descontado al confirmar la venta.

Procedimiento. Volví a Productos desde el comprobante y localicé el producto vendido. Repetí la consulta del producto 5016 en Postman y comparé el valor con la lectura anterior.

Resultado observado. La vista y la API mostraron 8 unidades. La diferencia respecto a las 10 iniciales coincide con las 2 unidades de la venta.



Figura 23. Productos muestra el stock actualizado a 8 unidades.

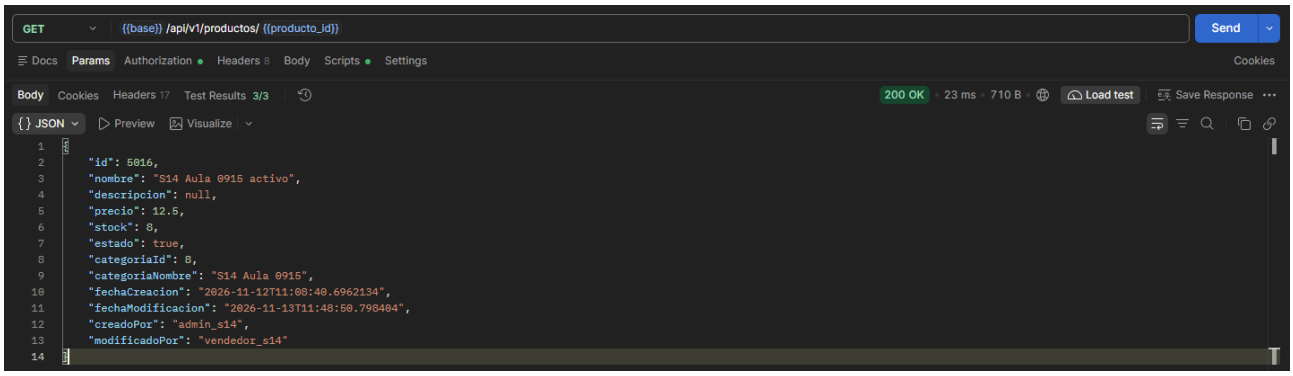


Figura 24. La respuesta de Postman confirma el stock de 8 unidades.

Matriz de integración I-04 Reporte

Producto vendido en el ranking

Objetivo. Comprobar que la venta registrada alimenta el reporte de productos más vendidos.

Procedimiento. En la sesión ADMIN consulté el ranking de la aplicación y envié GET /api/v1/reportes/productos-mas-vendidos en Postman. Localicé la fila del producto del ensayo y la comparé con el comprobante.

Resultado observado. S14 Aula 0915 activo figuró con 2 unidades y S/ 25,00 tanto en el ranking de la aplicación como en la respuesta de la API.

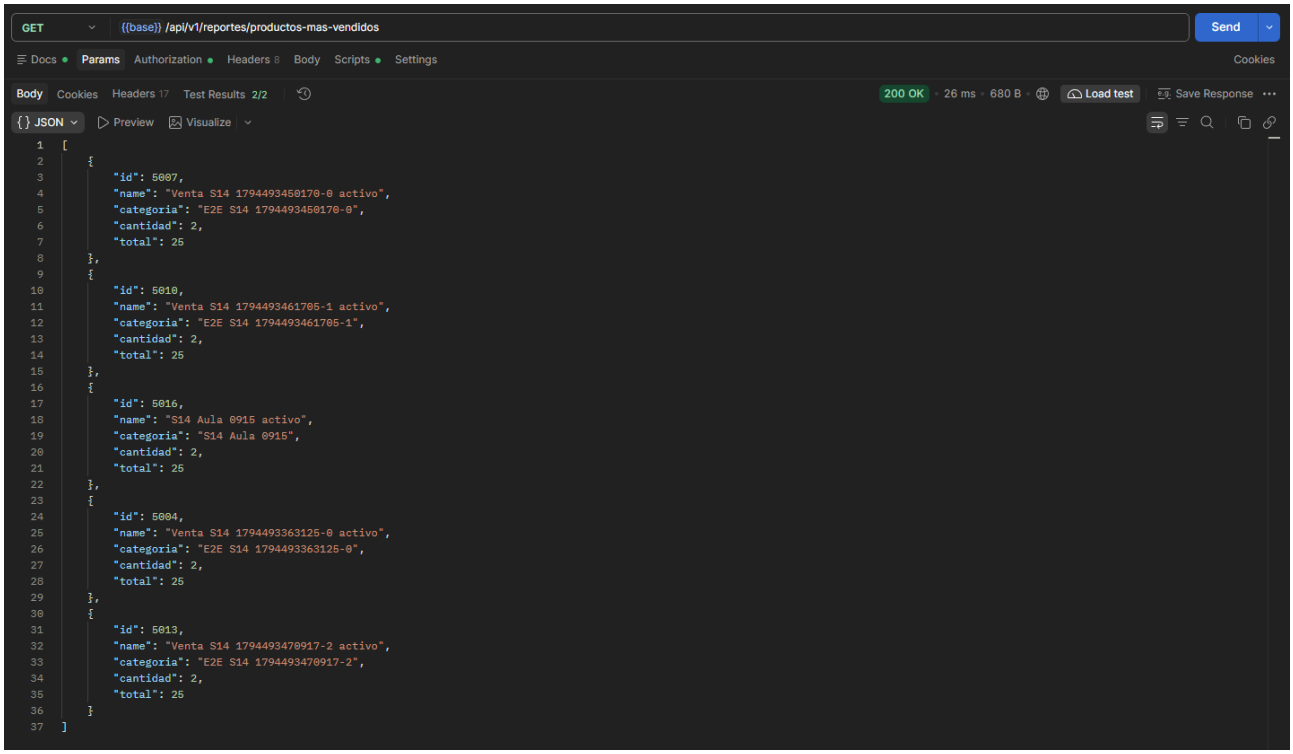


Productos más vendidos

Unidades e importes de las ventas registradas.

Producto	Categoría	Unidades vendidas	Monto
Venta S14 1794493450170-0 activo	E2E S14 1794493450170-0	2	S/ 25.00
Venta S14 1794493461705-1 activo	E2E S14 1794493461705-1	2	S/ 25.00
S14 Aula 0915 activo	S14 Aula 0915	2	S/ 25.00
Venta S14 1794493363125-0 activo	E2E S14 1794493363125-0	2	S/ 25.00

Figura 25. Ranking de la aplicación con las unidades y el importe.



GET {{base}} /api/v1/reportes/productos-mas-vendidos Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body Cookies Headers 17 Test Results 2/2 200 OK 26 ms 680 B Load test Save Response

{ } JSON Preview Visualize


```
[
    {
        "id": 5007,
        "name": "Venta S14 1794493450170-0 activo",
        "categoria": "E2E S14 1794493450170-0",
        "cantidad": 2,
        "total": 25
    },
    {
        "id": 5010,
        "name": "Venta S14 1794493461705-1 activo",
        "categoria": "E2E S14 1794493461705-1",
        "cantidad": 2,
        "total": 25
    },
    {
        "id": 5016,
        "name": "S14 Aula 0915 activo",
        "categoria": "S14 Aula 0915",
        "cantidad": 2,
        "total": 25
    },
    {
        "id": 5004,
        "name": "Venta S14 1794493363125-0 activo",
        "categoria": "E2E S14 1794493363125-0",
        "cantidad": 2,
        "total": 25
    },
    {
        "id": 5013,
        "name": "Venta S14 1794493470917-2 activo",
        "categoria": "E2E S14 1794493470917-2",
        "cantidad": 2,
        "total": 25
    }
]
```


Figura 26. Respuesta de Postman con el producto vendido, cantidad 2 y total 25.

Matriz de integración I-05 Bitácora

Evento de venta y acceso por rol

Objetivo. Correlacionar la venta con su evento de auditoría y comprobar que la bitácora exige el permiso correspondiente.

Procedimiento. Consulté GET /api/v1/auditoria como ADMIN, filtrando por VENTA_REGISTRADA y vendedor_s14. Comparé entidadId y requestId con la venta de I-03. Repetí la consulta con el token del vendedor.

Resultado observado. El evento identifica la venta 22221, el usuario vendedor_s14 y el requestId ce0c94d5. Con el vendedor, la API devolvió el HTTP 403 esperado. Esa captura muestra 0/1 en Postman y el cuerpo confirma la denegación de acceso.

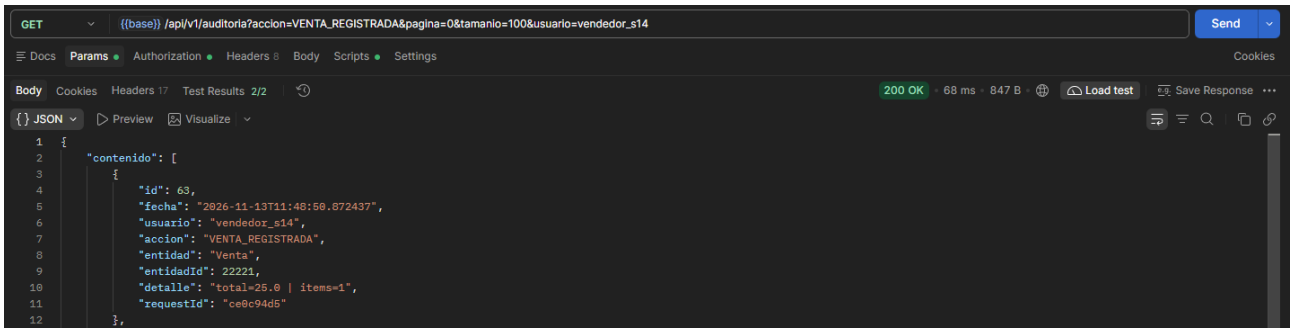


Figura 27. Evento VENTA_REGISTRADA con el ID de venta y el identificador de solicitud.



Figura 28. Respuesta HTTP 403 de la consulta de auditoría con el rol vendedor.

Matriz de integración I-05 Preparación del rollback

Estado antes de la venta rechazada

Objetivo. Obtener las lecturas que servirán para detectar un descuento parcial o un evento indebido.

Procedimiento. Consulté por separado los productos 5016 y 5018. Después filtré la bitácora por VENTA_REGISTRADA y anoté el campo totalElementos antes de enviar la venta.

Resultado observado. El producto activo tenía 8 unidades, el agotado tenía 0 y la bitácora registraba 5 eventos de venta.

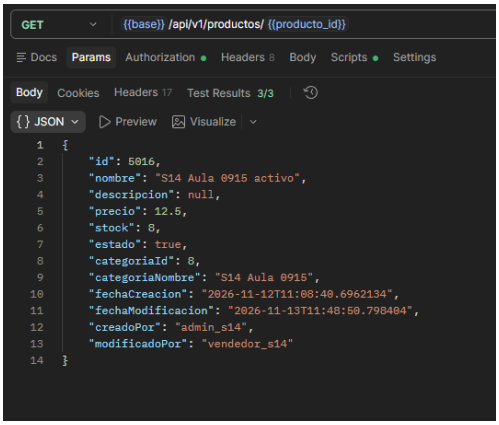


Figura 29. Stock del producto activo antes del intento.

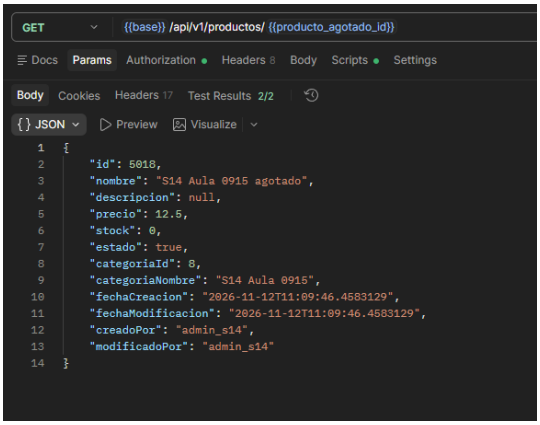


Figura 30. Stock 0 del producto activo que se usó para provocar el rechazo.

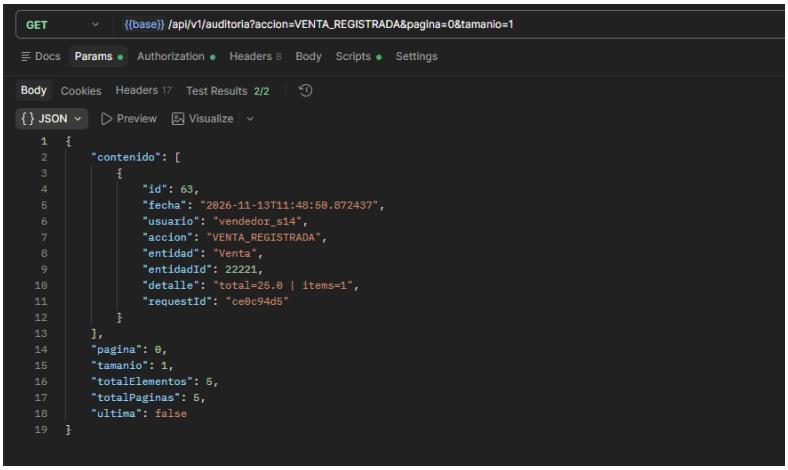


Figura 31. Cinco eventos de venta antes del intento.

Matriz de integración I-05 Rechazo y rollback

Venta rechazada sin efectos parciales

Objetivo. Verificar que al fallar una línea se revierte toda la operación y se conserva la bitácora de ventas confirmadas.

Procedimiento. Envié POST /api/v1/ventas como vendedor con dos líneas, una unidad del producto activo y una del agotado. Tras el rechazo repetí la consulta de stock y el conteo de VENTA_REGISTRADA.

Resultado observado. La API devolvió HTTP 409 por stock insuficiente. El activo conservó sus 8 unidades y el total siguió en 5. La evidencia verifica tanto el stock como la ausencia de un nuevo evento de venta.

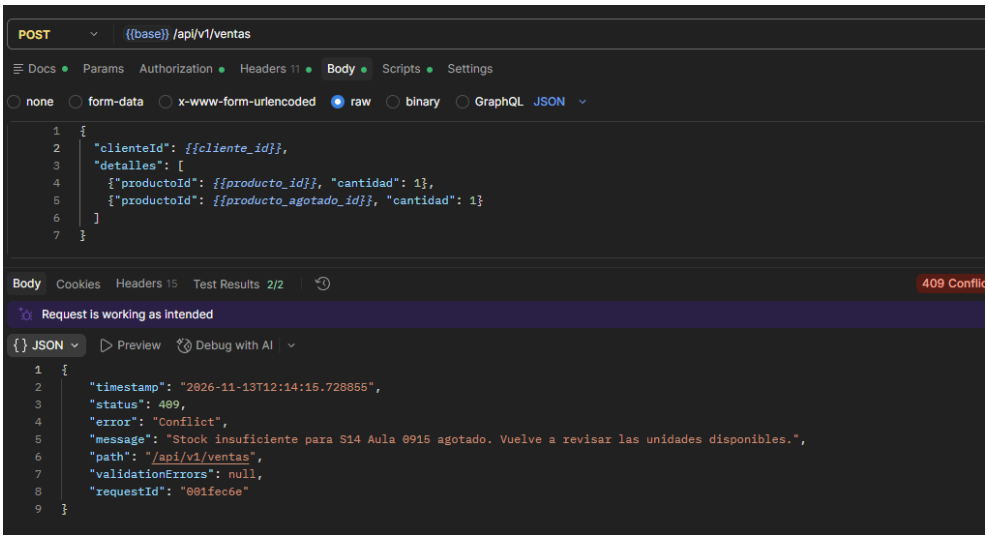


Figura 32. POST de dos líneas rechazado con HTTP 409 y requestId 001fec6e.

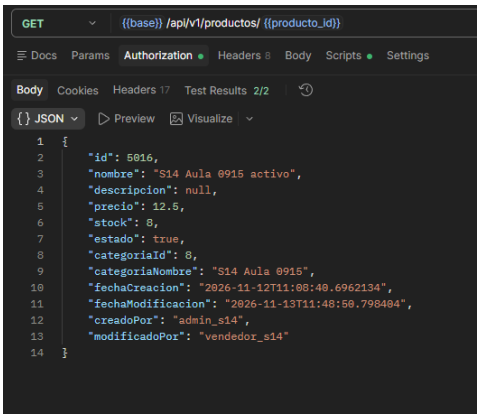


Figura 33. Stock intacto después del rechazo.

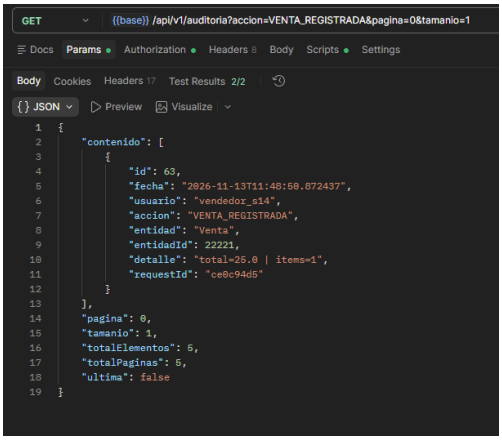


Figura 34. El total de eventos VENTA_REGISTRADA se conserva en 5.

Matriz de integración I-06 Precio y autor

Edición de precio

Objetivo. Comprobar que una modificación de catálogo conserva el autor de la operación y actualiza el precio.

Procedimiento. Como administrador abrí el producto 5016 en Productos, cambié el precio de S/ 12,50 a S/ 15,00 y guardé. Volví a abrir la edición y consulté el detalle en Postman.

Resultado observado. La vista mostró Modificado por admin_s14. El detalle devolvió precio 15, stock 8 y modificadoPor admin_s14.

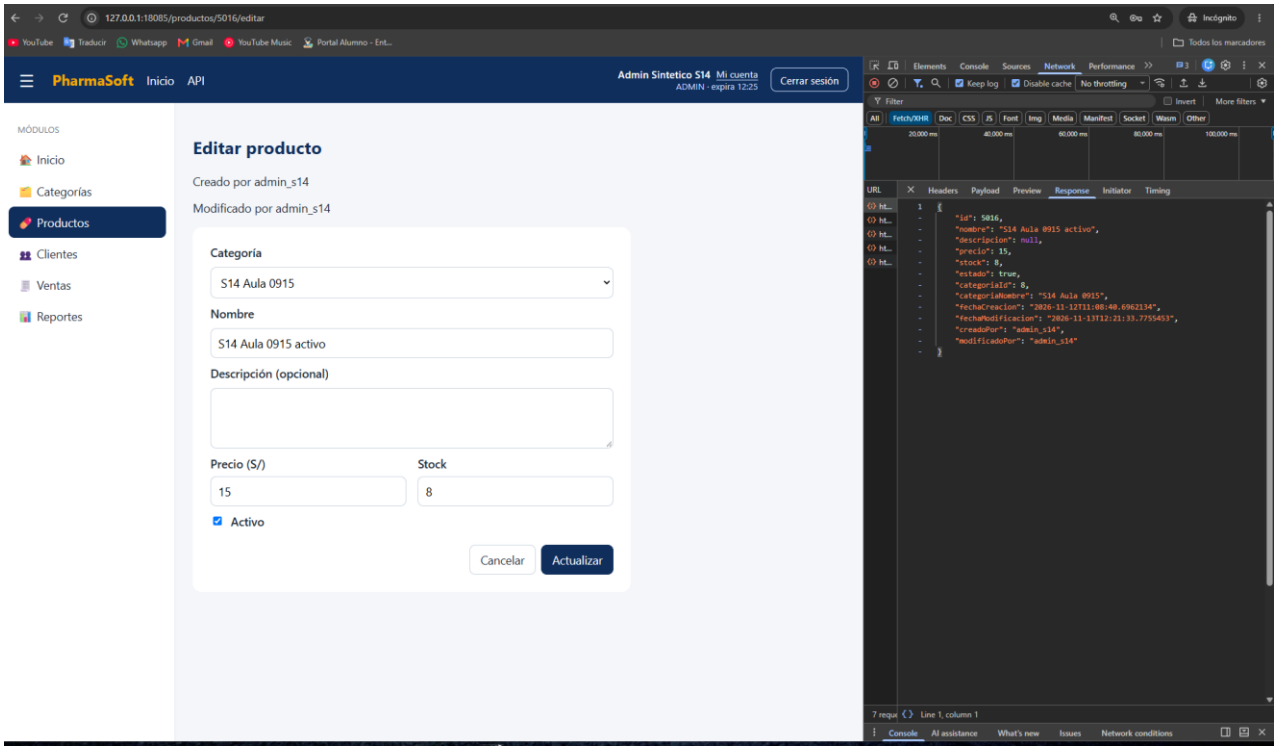


Figura 35. Edición del producto con el nuevo precio y el autor de la modificación.

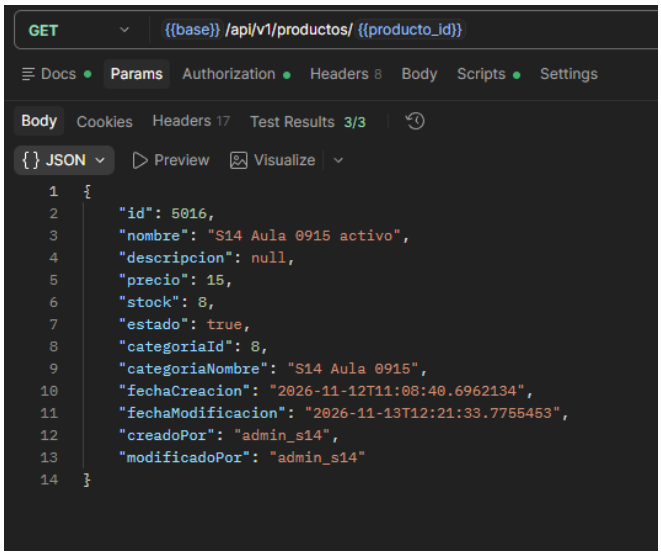


Figura 36. Detalle del producto 5016 con precio 15 y modificadoPor admin_s14.

Matriz de integración I-06 Evento de precio

Registro de la modificación

Objetivo. Vincular el cambio de precio con un evento que conserve el valor anterior, el nuevo y el autor.

Procedimiento. Filtré la bitácora por PRECIO_MODIFICADO y admin_s14. Localicé el evento del producto 5016 y comparé su requestId con el del PUT que guardó el cambio.

Resultado observado. El evento registró el paso de 12,50 a 15,00, el autor admin_s14 y el identificador fea9cebf. El mismo identificador aparece en el PUT del registro del backend.

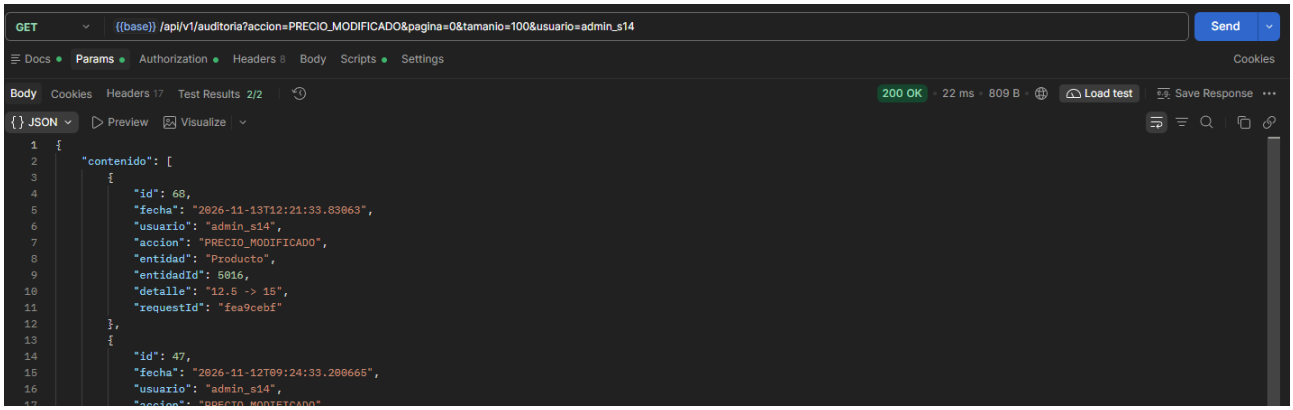


Figura 37. Consulta de auditoría con el evento y el identificador de la modificación.

Matriz de integración I-07 Sesión

Vencimiento y retorno a Nueva venta

Objetivo. Comprobar que al caducar la sesión se pide autenticación y se conserva el destino.

Procedimiento. Usé el laboratorio con una sesión de un minuto. Abrí Nueva venta como vendedor, esperé el vencimiento, revisé la redirección e inicié sesión otra vez.

Resultado observado. La aplicación mostró el aviso de expiración y la URL incluyó expirada=1 y returnUrl hacia /ventas/nueva. Tras autenticarme regresó a Nueva venta. El formulario se mostró vacío.

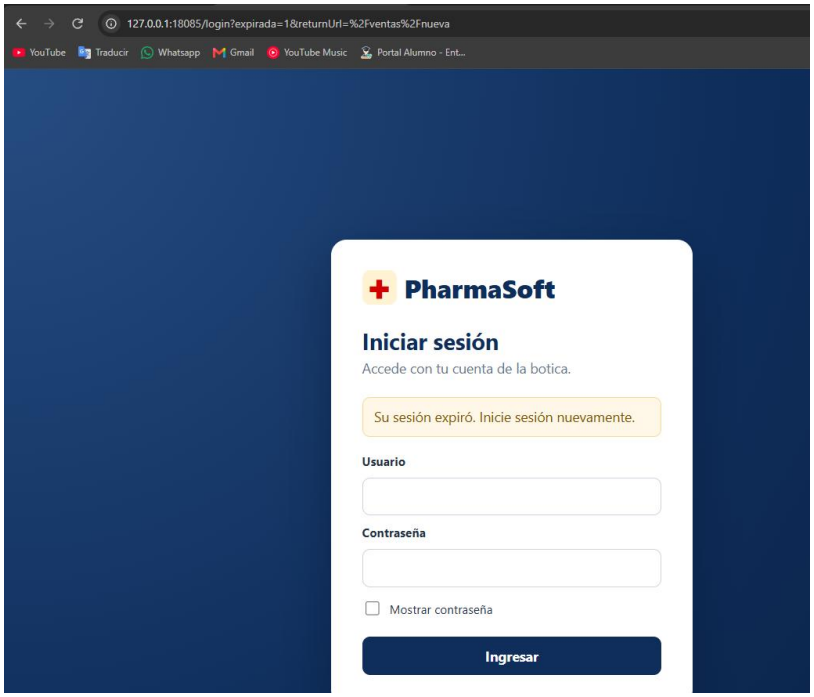


Figura 38. Inicio de sesión con el aviso de expiración y la ruta de retorno.

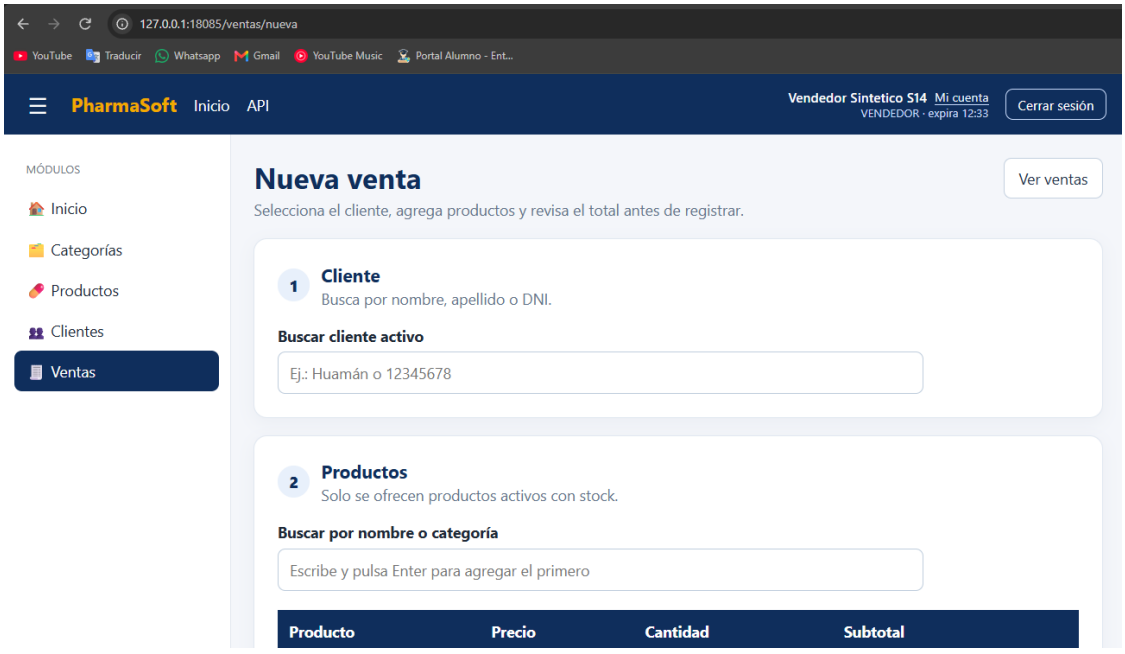


Figura 39. Nueva venta vuelve a abrirse tras iniciar sesión de nuevo.

Matriz de integración I-08 Filtros

Reinicio desde la página 5 de Ventas

Objetivo. Verificar que los filtros reinician la paginación y quedan representados en la URL.

Procedimiento. Abrí Ventas con pagina=4 y anoté el total inicial. Elegí Desde 01/11/2026 y Hasta 12/11/2026, pulsé Buscar y revisé el paginador, la dirección y la nueva solicitud.

Resultado observado. La vista pasó de Página 5 con 20 005 ventas a Página 1 con 641 resultados. La URL conservó desde=2026-11-01 y hasta=2026-11-12 con pagina=0.

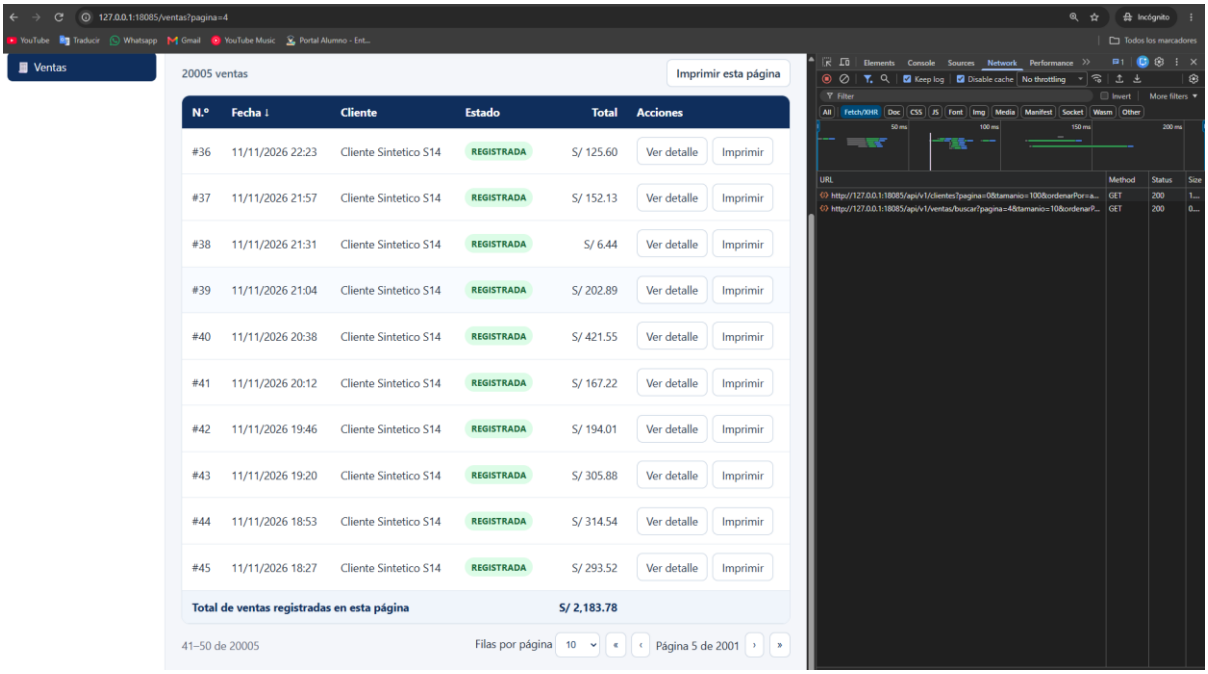


Figura 40. Página 5 de Ventas antes del cambio de filtro.

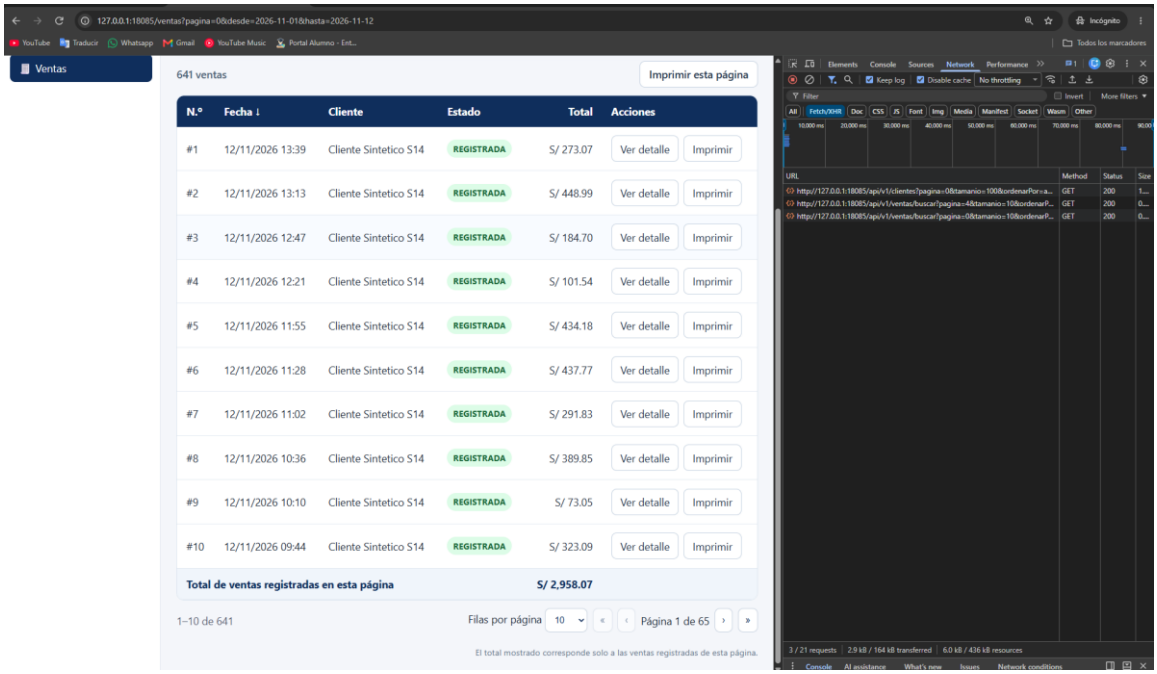


Figura 41. Página 1 de Ventas después de aplicar el rango.

P03 Playwright

Objetivo. Comprobar que los recorridos principales se repiten con el mismo resultado al usar la aplicación desde Chrome.

Procedimiento. Con el laboratorio activo, ejecuté npx.cmd playwright test --repeat-each=3 desde el frontend. Playwright corrió los siete casos con un trabajador y cero reintentos. Revisé la consola, el resumen JSON y el registro del backend.

Resultado inicial. La primera ejecución obtuvo 7/7, 6/7 y 7/7. La segunda repetición falló en el caso de retorno tras el vencimiento de sesión. El diagnóstico y la corrección quedaron registrados como INC-04.

Caso y comprobación	Primera	Segunda	Tercera
Ruta protegida. Redirige al login y conserva returnUrl.	Aprobó	Aprobó	Aprobó
Credenciales incorrectas. Devuelve 401 y genera LOGIN_FALLIDO.	Aprobó	Aprobó	Aprobó
Expiración. Vuelve a Nueva venta después de autenticarse otra vez.	Aprobó	Aprobó	Aprobó
Almacenero. No ve Ventas y se deniega el acceso directo.	Aprobó	Aprobó	Aprobó
Vendedor. Puede consultar Productos sin permisos de edición.	Aprobó	Aprobó	Aprobó
Paginación. Conserva la página al recargar y la reinicia al filtrar.	Aprobó	Aprobó	Aprobó
Venta. Comprueba stock, reporte, auditoría, precio y rollback.	Aprobó	Aprobó	Aprobó

Resultado final. Después del ajuste, se aprobaron 21 de 21 en 23,2 s. El backend registró cero respuestas HTTP 500 y el flujo de venta registró cero excepciones del navegador en las tres repeticiones.



Figura 42. Consola de la corrida final con 21 pruebas aprobadas.

P03 Casos automatizados

Cada escenario verifica una acción observable en la interfaz, en la API o en ambas. Los siete casos aprobaron en cada una de las tres repeticiones del informe final.

1. Ruta protegida. Abrí /ventas en un contexto sin sesión. La comprobación exigió que la aplicación enviara al login y guardara el destino original en returnUrl para poder retomarlo.

2. Credenciales incorrectas. Envié el formulario con un usuario de ensayo inexistente. El caso comprobó HTTP 401 y el mensaje de la interfaz. Después consultó la bitácora con ADMIN y comparó el identificador de LOGIN_FALLIDO con el de la respuesta.

3. Retorno y vencimiento. Entré como vendedor desde la ruta protegida, abrí Nueva venta y adelanté el reloj del navegador para vencer la sesión. El caso comprobó la URL de expiración y que un segundo inicio de sesión abriera de nuevo Nueva venta.

4. Permisos del almacenero. Inicié sesión como almacenero y comprobé que el menú ocultara Ventas. Abrí /ventas directamente y verifiqué que la aplicación mostrara Acceso denegado.

5. Permisos del vendedor. Abrí Productos como vendedor. La prueba exigió que se pudiera consultar la tabla y comprobó que no se ofrecieran las acciones Nuevo producto ni Editar.

6. Paginación y filtros. Abrí la segunda página de Productos, comprobé la solicitud al servidor y recargué el navegador. Luego apliqué el rango de fechas en la página 5 de Ventas y verifiqué el regreso a la página 1.

7. Flujo de venta. Playwright creó sus propios registros, confirmó una venta de dos unidades y verificó el comprobante, el stock y el reporte. También comprobó la auditoría, el cambio de precio y que el rechazo de una venta con stock insuficiente conservara el estado anterior.

La consola de la corrida y el informe HTML reúnen los resultados de estos escenarios. El adjunto de Playwright muestra el stock del flujo de venta.

P03 Informe HTML y stock

Informe de Playwright

Objetivo. Comprobar los totales de la ejecución y conservar un resultado visible del flujo principal.

Procedimiento. Abrí el informe con npx.cmd playwright show-report y revisé los contadores. En el caso de venta de la primera repetición consulté el adjunto Stock después de la venta.

Resultado observado. El informe muestra 21 aprobadas, 0 fallidas, 0 inestables y 0 omitidas. El adjunto muestra 8 unidades para el producto activo del flujo automatizado.

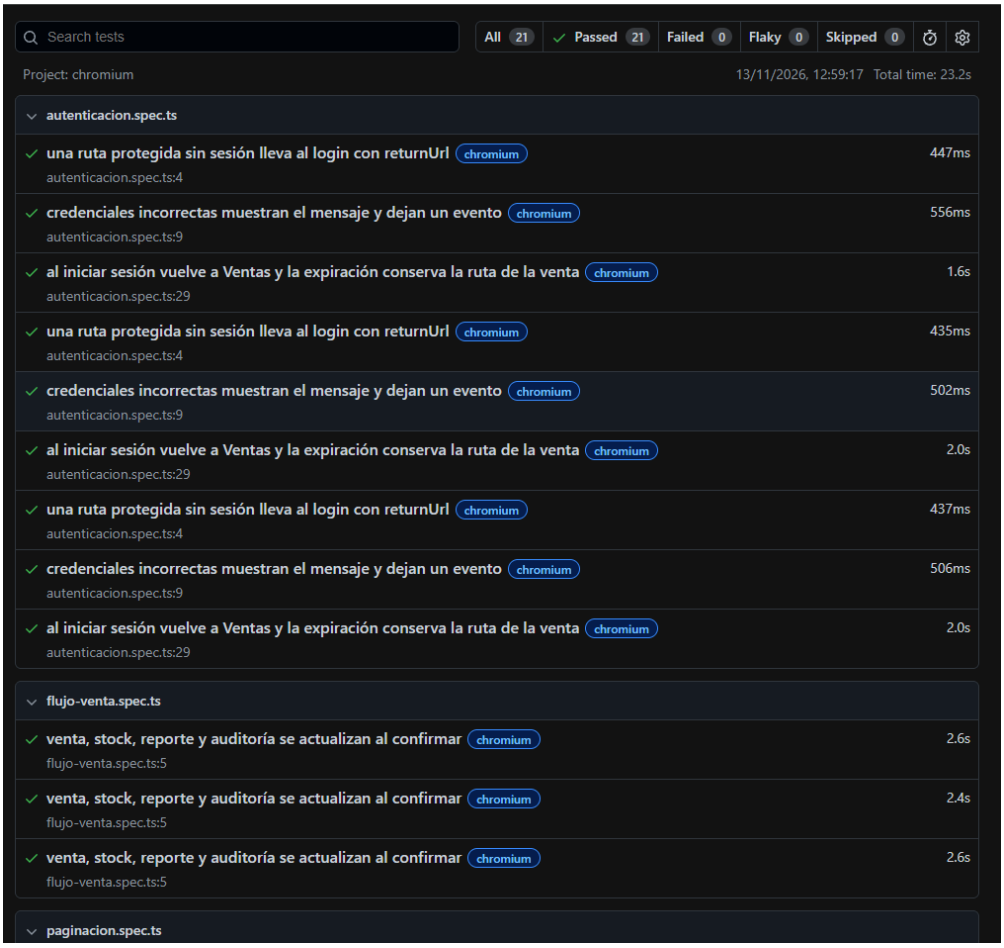


Figura 43. Informe HTML de la corrida final.

Nombre ↑	Categoría	Precio	Stock	Estado
Venta S14 1794592760957-0 activo	E2E S14 1794592760957-0	S/ 12.50	8	Activo
Venta S14 1794592760957-0 baja	E2E S14 1794592760957-0	S/ 12.50	10	Inactivo
Venta S14 1794592760957-0 sin stock	E2E S14 1794592760957-0	S/ 12.50	0	Activo

Figura 44. Adjunto de Playwright con la evidencia del stock después de la venta.

P04 Incidencias y cierre

Procedimiento. Registré los resultados iniciales, comparé la salida de Playwright con los registros de Nginx y del backend, y volví a ejecutar los casos después del ajuste. La tabla reúne las incidencias de la sesión y el estado de su verificación.

ID	Causa y corrección	Estado
INC-01	Nginx reenviaba el host sin el puerto original. Se conservó el host y el puerto de la solicitud. Frontend e9e768c.	Cerrada
INC-02	La aserción del comprobante omitía el parámetro nueva=1. Se ajustó la comprobación de la URL. Frontend 1c73f32.	Cerrada
INC-03	La prueba aislada de caché simulaba save. Se actualizó para simular saveAndFlush. Backend 242f390.	Cerrada
INC-04	La segunda repetición quedó en el login. Nginx registró 499 al cerrarse una solicitud pendiente. Se habilitó la reutilización de conexiones HTTP/1.1 entre Nginx y el backend. El caso aislado pasó 10/10 y P03 pasó 21/21.	Cerrada en el laboratorio

Versionamiento. Rama feature/integracion-estabilizacion-pacho en ambos proyectos. Referencias locales al cierre de las pruebas, backend 5c025ef y frontend 1c73f32. El ajuste de INC-04 y la actualización del informe están en el espacio de trabajo local.

Archivos de respaldo. El registro de línea base, la bitácora de incidencias, las capturas S14 y el resumen JSON de Playwright se encuentran en los dos proyectos.

Conclusión. Se comprobó la paginación en el servidor, la integración de los ocho casos y la trazabilidad de la venta y del cambio de precio. El intento de venta con un producto agotado conservó el stock y los eventos previos. Tras ajustar Nginx, los siete casos automatizados pasaron las tres repeticiones.